

SiPy Execution Logs as Reproducibility Infrastructure

Maurice HT Ling^{1,2*}¹HOHY PTE LTD, Singapore.²Newcastle Australia Institute of Higher Education, University of Newcastle, Australia.***Correspondence:**

Maurice HT Ling, Newcastle Australia Institute of Higher Education, University of Newcastle, Australia.

Received: 21 Aug 2026; **Accepted:** 22 Sep 2026; **Published:** 30 Sep 2026**Citation:** Maurice HT Ling. SiPy Execution Logs as Reproducibility Infrastructure. American J Pathol Res. 2026; 5(9): 1-3.**ABSTRACT**

Computational reproducibility requires more than retaining the final numerical results of an analysis. The computational environment, software versions, commands and execution sequence may all influence the results, therefore, form part of the computational record. Execution logs provide a straightforward mechanism for recording such information but conventional logs are generally passive records intended primarily for debugging or retrospective inspection. We describe an execution log framework implemented in SiPy, a lightweight statistical interface that coordinates external Python, R, Julia and shell scripts. The SiPy execution log records the SiPy version and release information, execution environment, interpreter versions, selected analytical Python packages, commands, results and timestamps. Five operations provide a complete execution record workflow: saving a log, reading a log, extracting its history, comparing the recorded environment with the current environment, and replaying the recorded commands to compare results. This transforms the execution log from a passive record into an executable provenance record that can be used to investigate reproducibility.

Keywords

Computational reproducibility, Provenance, Execution logging, Statistical computing, Scientific software, SiPy.

Introduction

Computational research increasingly depends on software environments whose components may change independently of the scientific analysis. Software dependencies can change in ways that affect computational outputs, while differences in software packaging, installation and execution can make computational findings difficult to reproduce [1,2]. Therefore, preserving a computational result requires more than retaining the final output. Sandve et al. [1] and Peng [3] identified keeping track of how every result was produced, recording exact versions of external programs and preserving intermediate results among the basic practices required for reproducible computational research. Computational provenance then provides a framework to record the activities, entities and processes involved in an analysis and can support inspection and replay of computational workflows [2,4]. These practices complement broader recommendations for scientific software development, including version control, automated testing and systematic management of computational software [5]. Computational notebooks have provided another important

approach to recording executable analyses by combining code, results and narrative documentation [6]. However, preserving a notebook alone does not necessarily ensure reproducibility, because its execution may depend on external data, software dependencies and computational environments that are not contained within the notebook [6,7].

These observations motivate a relatively simple question: can an ordinary execution log be designed so that it does more than record what happened? SiPy [8] provides a suitable environment for exploring this question. SiPy is a lightweight statistical interface written in Python that coordinates external scripts written in Python, R, Julia and shell languages [9]. Rather than embedding multiple programming languages into a single computational environment, SiPy maintains explicit execution boundaries and treats external scripts as first class computational objects. An execution log therefore provides a natural place to record both the analysis history and the computational environment in which that history was executed. Here, I describe the execution log framework in SiPy and demonstrate how it extends ordinary session logging into a mechanism for environment comparison, analysis recovery and result replay.

Execution Log Structure

SiPy execution logs are formatted as initialization file format to remain human readable while also providing structured information that can be parsed programmatically. There are 8 sections in the execution log file: (i) a log begins with a [sipy] section containing the SiPy version, codename and release date. (ii) [environment] section records the execution configuration. This includes the current working directory, SiPy directory, executable paths for Python, R and Julia, plugin configuration, command separator, timing and verbosity settings. Importantly, the log also records whether each interpreter is embedded within the SiPy distribution or externally provided. (iii) [log_generation_timestamp] section records both a UTC epoch value and local time. (iv) [history] section contains the commands executed during the session, indexed sequentially. (v) [result] section stores the output associated with each command. (vi) [timestamp] section records execution times for individual history entries. (vii) the [system_information] records operating system information, processor architecture, Python version and build information, CPU count, platform and the versions of R and Julia. (viii) [important_python_packages] section records selected Python packages considered relevant to analytical execution.

This structure deliberately separates four related but distinct elements of computational provenance, namely environment, history, result and time. Such explicit recording of execution context and workflow history is consistent with established approaches to computational provenance and workflow reproducibility [10].

Execution Log Operations

Five log operations currently form the principal execution log interface. These are

1. “log save” to create a persistent execution log from the current SiPy session. Therefore, the log captures the computational state and history rather than requiring the researcher to reconstruct these details retrospectively.
2. “log read” to display an existing execution log, allowing the recorded environment, commands, results and timestamps to be inspected.
3. “log extract history” to retrieve the command history contained in the log. This provides a direct route from the persistent record back to the executable analysis sequence.
4. “log compare system” to compare the current computational environment against the environment stored in the log. The comparison is performed at the level of individual recorded attributes and reports each as either Match or Mismatch.
5. “log compare result” to extract the recorded commands in order, re-executes them and compares the resulting outputs against the results stored in the original execution log.

Together, these operations provide a simple execution lifecycle: save → read → extract → compare environment → replay and compare results. The significance of this arrangement is that the log is both a record and a source for subsequent computation. This combination of retrospective provenance and executable workflow information is consistent with approaches developed for

reproducible computational workflows [10,11].

Discussion

The principal design concept of the SiPy implementation is the transition from a passive execution log to an executable provenance record. A conventional log answers the question, what happened? A provenance record adds to the question by asking, under what computational conditions did it happen? An executable provenance record adds a further question as, can the recorded sequence be executed again, and what happens when it is?

This distinction is consistent with the broader literature distinguishing documentation of computational procedures from active testing of their reproducibility. Computational reproducibility can be supported through approaches ranging from detailed documentation and versioned software to workflow systems and encapsulated computational environments [7,10]. More recent work has explicitly proposed reproducibility tests that compare computational executions rather than relying solely on the presence of workflow descriptions or environmental metadata [11]. Santana-Pérez et al. [4] demonstrated the importance of preserving execution environments alongside workflow information. Similarly, computational reproducibility reviews have identified software dependencies and their versions as important determinants of whether an analysis can be reproduced [7]. Consequently, comparing the recorded and current environments provides useful information about potential sources of computational variation, even when the analysis itself can subsequently be replayed.

Environment comparison and result comparison therefore address different aspects of reproducibility. An environment mismatch identifies a potential source of computational variation but does not establish that a result has changed. Conversely, agreement between results does not establish that the computational environments were identical. Recent work on formal reproducibility testing similarly emphasizes the value of explicitly testing computational executions and identifying numerical differences between workflow runs [11].

SiPy does not attempt to solve the complete provenance problem. Rather, it integrates a subset of these principles directly into a statistical execution environment. This is particularly relevant to lightweight computational analyses. A researcher performing a small statistical calculation may not need a formal workflow management system, container orchestration, or a dedicated provenance database. Nevertheless, the analysis may still benefit from retaining exactly what was executed, the environment in which it was executed and the resulting outputs. The SiPy execution log provides this layer with relatively little additional complexity.

Limitations

The current implementation should not be interpreted as a complete reproducibility solution. Firstly, the log records only selected components of the computational environment and therefore does not preserve every dependency required to reconstruct that environment. Secondly, external scripts are not

necessarily embedded in the log. Thirdly, external input data are not automatically incorporated into the execution record. Fourthly, stochastic analyses may require explicit recording of random seeds as Sandve et al. [1] specifically identify recording random seeds as a reproducibility requirement for analyses involving randomness. Finally, result comparison currently operates on the recorded outputs. Numerical computations can exhibit small differences across software versions or hardware environments even when the underlying analysis is substantively unchanged. Hence, future versions may include explicit numerical tolerances and more structured result comparison. These limitations define the intended scope of the current implementation. The execution log is not presented as a replacement for containerization, workflow management, version control, archival of data or comprehensive environment capture [7,10]. Instead, it provides a lightweight reproducibility mechanism at the level of an individual statistical computing session.

Conclusion

SiPy introduces an execution log framework that extends conventional session logging into a lightweight reproducibility mechanism. The log records the SiPy version, computational environment, interpreter and package versions, command history, results and timestamps. Five operations allow the record to be saved, inspected, recovered, compared with a current environment and replayed for result comparison. In doing so, SiPy provides a practical bridge between informal execution history and fully managed reproducible computational workflows. Rather than attempting to reproduce an entire computational infrastructure, the approach provides a lightweight mechanism for recording execution context, recovering computational history and testing whether replay produces the recorded results.

Supplementary Materials

SiPy execution log is implemented in SiPy 0.11.0 (<https://bit.ly/SiPy-0110>). Source codes of SiPy can be found at <https://github.com/mauriceling/sipy>.

References

1. Sandve GK, Nekrutenko A, Taylor J, et al. Ten Simple Rules for Reproducible Computational Research. *PLoS Comput Biol*. 2013; 9: e1003285.
2. Kanwal S, Khan FZ, Lonie A, et al. Investigating reproducibility and tracking provenance - A genomic workflow case study. *BMC Bioinformatics*. 2017; 18: 337.
3. Peng RD. Reproducible Research in Computational Science. *Science*. 2011; 334: 1226.
4. Santana-Perez I, Ferreira Da Silva R, Rynge M, et al. Reproducibility of execution environments in computational science using Semantics and Clouds. *Future Generation Computer Systems*. 2017; 67: 354-367.
5. Wilson G, Aruliah DA, Brown CT, et al. Best Practices for Scientific Computing. *PLoS Biol*. 2014; 12: e1001745.
6. Rule A, Birmingham A, Zuniga C, et al. Ten simple rules for writing and sharing computational analyses in Jupyter Notebooks. *PLoS Comput Biol*. 2019; 15: e1007007.
7. Piccolo SR, Frampton MB. Tools and techniques for computational reproducibility. *Gigascience*. 2016; 5: 30.
8. Tan NT, Mugundhan M, Liu T, et al. SiPy - Bringing Python and R to the End-User in a Plugin-Extensible System. *Medicon Medical Sciences*. 2025; 8: 32-41.
9. Ling MH. SiPy 0.8.0 on the Three Legs of Python, R, and Julia. *Acta Scientific Computer Sciences*. 2026; 8: 01-09.
10. Khan FZ, Soiland-Reyes S, Sinnott RO, et al. Sharing interoperable workflow provenance: A review of best practices and their practical application in CWLProv. *Gigascience*. 2019; 8: giz095.
11. Pritchard NJ, Wicenc A. Formal definition and implementation of reproducibility tenets for computational workflows. *Future Generation Computer Systems*. 2025; 166: 107684.